MacZine
THE MACTECH SOLUTIONS NEWSLETTER
ISSUE Nº 033 · SEPTEMBER 2026 · MACTECHSOLUTIONSLLC.COM/MACZINE

PLATFORM SPOTLIGHT · AI & AUTOMATION

IBE Decides an AI Agent's Authority Before It Acts

IBE governs what an AI agent may do with no LLM in the decision path - a deterministic autonomy gate that certifies or refuses every action, in writing.

BY **MAXINE** READING TIME · 6 MIN AI-AUTOMATION · GOVERNANCE · DEVSECOPS · MACTECH-SUITE

defense contractor that has spent two years proving to a C3PAO that every access decision on its network is logged, reviewable, and owned by a named person does not get to wave that discipline away the day it turns an AI agent loose on its own repository. Give a coding agent commit access, deploy access, or the ability to touch a build pipeline, and you have created exactly the kind of decision point NIST 800-171 spent 110 requirements teaching you to control - except this one moves at the speed of an API call, keeps no memory of why it did what it did unless something makes it, and can in principle be talked into almost anything by a sufficiently patient prompt. Whether AI agents are useful is not in serious doubt; this article was drafted by one, dispatched from a commission issue, on a repository governed by exactly the tool this piece describes. The harder question a defense contractor actually has to answer is whether "useful" and "authorized" are the same thing. Most agent tooling on the market only answers the first.

IBE, short for Intent Bound Execution, is MacTech's answer to the second question, and it draws the line in a place that sounds obvious in a compliance shop and is almost never built that way in an engineering one: authority to act gets decided before the action runs, by something that cannot be talked out of its answer, and the decision leaves a signed record behind whether it says yes or no. MacTech Files Three Provisional Patents Built on Proof, Not Trust covered IBE's capability ratchet as one of three inventions built on the same worldview - determinism you can verify instead of an operator's word. This piece stays with IBE alone, and with the narrower question a program office actually has to answer before it hands an agent the keys: not whether the engineering is clever, but whether the company can still pass an assessment with the agent inside the boundary.

The gate that never asks the model

IBE's kernel decides whether a proposed agent action is authorized by running it through eight conjunctive gates - all eight have to clear, none can be waived by a convincing argument, because there is no argument to make. The doctrine, stated plainly in the code, is that no large language model sits anywhere in the decision path. `decide()` is a pure function: no clock, no randomness, nothing a prompt can move. An agent does not get to explain its way past a gate the way it might talk a human reviewer into an exception on a busy afternoon.

The output is not a log line somebody has to go find later. Every decision produces a signed acceptance certificate or a signed refusal certificate that names the specific gate that failed. For an assessor asking who authorized an agent to touch a given system and on what basis, that certificate is the answer, generated at the moment of the decision instead of reconstructed afterward from scattered logs. A refusal is not a silent no; it is evidence with a reason attached, which is the property compliance programs spend most of their documentation budget trying to manufacture after the fact.

Authority that spends itself

The harder problem in any permission system is what happens after the grant is issued. A stolen API key or a leaked token is usually good until somebody notices and rotates it - a window measured in hours or days, during which the credential works exactly as well for an attacker as for the agent it was meant for.

IBE's capability ratchet closes that window by construction rather than by response. Authority for a specific action derives from a one-way hash ratchet seeded by the intent and the engineering-model version behind it, and the holder erases each key the moment it is spent. A captured token cannot be replayed, even inside what would otherwise be its valid window, because the key that would replay it no longer exists anywhere to erase. The verifier's side of the bargain is just as deliberately thin: it keeps only the next expected index and a rolling accumulator, a small, publishable, order-binding commitment to exactly which grants have been spent - so a third party can audit that history without ever holding the keys themselves. MacTech is specific about what this is not: macaroons and Biscuit tokens attenuate authority but are not forward-secure, a bearer JWT is replayable for its whole lifetime, Signal's ratchet protects message keys rather than execution authority, and nullifier spent-sets enforce single-use without binding to a model version at all. The combination is the invention; none of the pieces alone would be.

A model swap kills every outstanding grant

FIELD COPY
ISSUE Nº 033
AI AUTOMATION

Bump the engineering model one revision, and the prior run's evidence is refused, by name, in a signed refusal certificate.

That line describes a killswitch nobody has to pull. Because every grant is seeded by the engineering-model version in force when it was issued, changing that version reseeds the entire ratchet at once. Every outstanding token tied to the old version stops working - not eventually, not after somebody remembers to run a revocation script, but the instant the version changes. Across 73 of 73 end-to-end tests, MacTech has demonstrated the two halves of that claim together: a benign change under the current model is accepted with a signed certificate, and the identical change re-run one model revision later is refused with a signed certificate naming the mismatch. For a security team, the operational consequence is that swapping the model a coding agent runs on - something that will happen routinely as the underlying tools improve - is itself the revocation event, not a separate task that has to make it onto somebody's checklist.

Where it sits in the estate

None of this makes IBE a code reviewer, and MacTech does not sell it as one. The kernel decides standing, not quality - whether an agent had the authority to propose a given action, not whether the action was the right call. That judgment, on this piece included, still belongs to a human reading a diff before a merge button gets pressed; IBE's certificates are what that human, and later an assessor, get to check the decision against instead of trusting that it happened.

The same governed-agent pattern runs through the rest of what MacTech builds and operates: the MacTech Suite's deploy operations auto-merge a narrow, pre-approved class of crash fixes and nothing wider; the Press Room that commissioned this article turns an editor's click into a GitHub issue, an agent's draft, and a human's merge, never the other way around. IBE is the piece of that pattern built to be handed to a defense contractor rather than kept in-house - a way to let an agent move at agent speed inside a boundary an assessor can actually verify, one signed certificate at a time. Programs weighing how far to let agent tooling reach into a CUI-adjacent build pipeline can start that conversation at /readiness or get in touch directly. ◆

8

CONJUNCTIVE GATES A PROPOSED AI AGENT ACTION MUST CLEAR - NO LLM ANYWHERE IN THE DECISION PATH

73/73

END-TO-END TESTS DEMONSTRATING A MODEL-VERSION BUMP REFUSES EVERY OUTSTANDING GRANT, BY NAME, IN A SIGNED CERTIFICATE

0

REVOCATION CALLS NEEDED TO KILL A STALE OR COMPROMISED GRANT - A MODEL SWAP DOES IT AUTOMATICALLY

◆ WHAT IBE IS NOT

IBE does not decide whether an agent's proposed change is a good idea. It decides whether the agent has standing to make it. Judgment on the change itself - is this the right fix, the right design, the right words for this newsletter - still waits for a human to read the diff.